

Ex no: 5(c)	Creating a Custom Docker Image and Uploading It to Docker Hub
Date:	

Aim:

To create a custom Docker image from a simple application, test it locally, and upload (push) the image to Docker Hub for sharing and deployment.

Requirements / Tools

- PC with Docker installed

Docker Desktop (Windows/Mac)

Docker Engine (Linux)

- Internet connection
- Docker Hub account
- Text editor

Notepad

Visual Studio Code (VS Code)

- Terminal

PowerShell

Command Prompt

Bash

- A simple application (Python or Node.js)

Option 1: Python Application

myapp/

|— app.py

|— Dockerfile

|— .dockerignore

app.py

```
print("Hello Docker Hub!!!")
```

Dockerfile

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```

.dockerignore

```
__pycache__/
```

```
*.pyc
```

```
*.log
```

Option 2: Node.js Application

```
myapp-node/
```

```
├── package.json
```

```
├── index.js
```

```
├── Dockerfile
```

```
└── .dockerignore
```

package.json

```
{  
  
  "name": "myapp-node",  
  
  "version": "1.0.0",  
  
  "main": "index.js"  
  
}
```

index.js

```
console.log("Hello from Docker!");
```

Dockerfile

FROM node:18-slim

WORKDIR /app

COPY . .

CMD ["node", "index.js"]

.dockerignore

node_modules

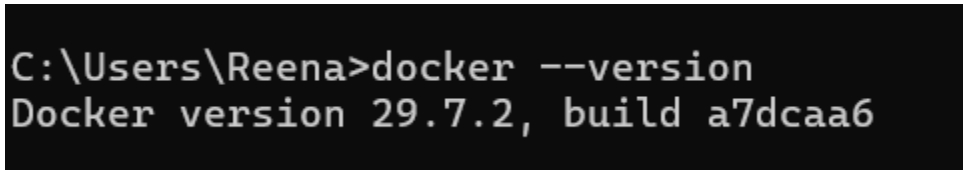
npm-debug.log

Procedure

Step 1: Verify Docker Installation

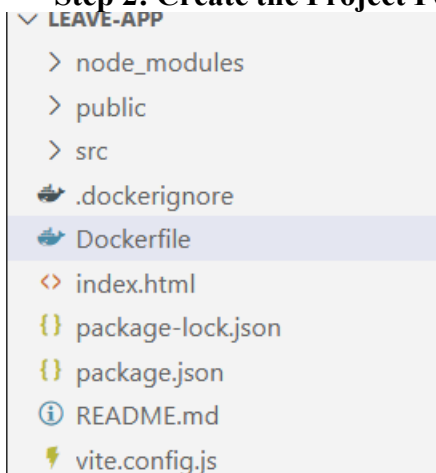
Open PowerShell, Command Prompt, or Bash and run:

docker --version



```
C:\Users\Reena>docker --version
Docker version 29.7.2, build a7dcaa6
```

Step 2: Create the Project Folder and Files



Step 3: Build the Docker Image Locally

Navigate to the project folder:

```
cd myapp
```

```
docker build -t my-simple-
```

```
image1 .
```

```
PS C:\Users\Reena\Desktop\leave-app> docker build -t leave-app .
[+] Building 25.1s (16/16) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 351B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 117B
=> [build 1/6] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
=> => sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 445B / 445B
=> => sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 1.26MB / 1.26MB
=> => sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 43.23MB / 43.23MB
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb
=> => extracting sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287
=> => extracting sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122
```

Step 4: Verify the Image List available Docker images

docker images

```
C:\Users\Reena>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
docker/welcome-to-docker:latest	c4d56c24da4f	22.2MB	6.03MB	U
hello-world:latest	5dd0d3e6e255	25.9kB	9.49kB	

Step 5: Run the Image Locally

Command:

```
docker run --rm my-simple-image1
```

```
PS C:\Users\Reena\Downloads\Workspace> docker run hello-world
Hello, World!
```

Step 6: Create a Repository on Docker Hub

1. Open Docker Hub.
2. Sign in to your account.
3. Click **Create Repository**.

4. Enter repository name:

my-simple-image

5. Select:

Public

6. Click **Create**.

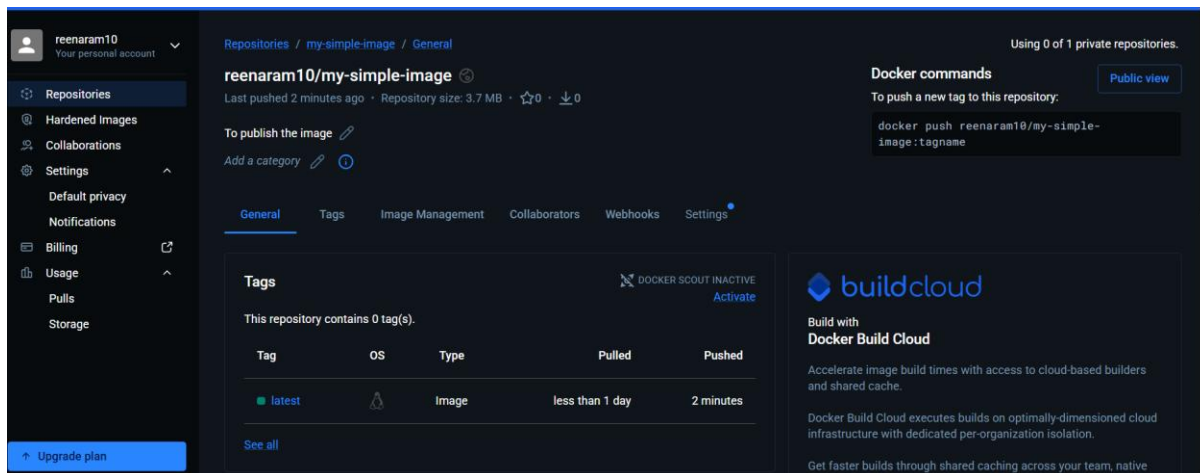
Step 7: Log in to Docker Hub

Open the terminal and run:

docker login

Enter your:

- Docker Hub Username
- Docker Hub Password



Step 8: Tag the Docker Image

If the image was built as myapp:1.0, tag it with your Docker Hub username:

```
docker tag my-simple-image1 <dockerhub-username>/my-simple-image1
```

```
PS C:\Users\Reena\Desktop\my-simple-app> docker tag my-simple-app reenaram10/my-simple-image:latest
```

Step 9: Push the Image to Docker Hub

Upload the image:

```
docker push <dockerhub-username>/my-simple-image1
```

```
PS C:\Users\Reena\Desktop\my-simple-app> docker push reenaram10/my-simple-image:latest
The push refers to repository [docker.io/reenaram10/my-simple-image]
55afa1ecc21d: Pushed
6dd07ffb2d03: Pushed
44136fa355b3: Pushed
latest: digest: sha256:791177918eb41cef7178491b4ec3c33db58ac6963643fd033d3dea6fc065322f size: 855
PS C:\Users\Reena\Desktop\my-simple-app>
```

Step 10: Verify the Image from Docker Hub

Pull the image from Docker Hub:

```
docker pull <dockerhub-username>/my-simple-image1
```

```
PS C:\Users\Reena\Desktop\my-simple-app> docker pull reenaram10/my-simple-image:latest
latest: Pulling from reenaram10/my-simple-image
44136fa355b3: Already exists
6dd07ffb2d03: Download complete
Digest: sha256:791177918eb41cef7178491b4ec3c33db58ac6963643fd033d3dea6fc065322f
Status: Downloaded newer image for reenaram10/my-simple-image:latest
docker.io/reenaram10/my-simple-image:latest
```

Rubrics:

Problem Understanding & approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Time Management (5)	Viva (5)	Total (50)

Result:

A custom Docker image was successfully created from a simple application, tested locally, tagged, uploaded to Docker Hub, and verified by pulling and running it from the repository



Rubrics:

Problem Understanding & approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Time Management (5)	Viva (5)	Total (50)

Result:

A custom Docker image was successfully created from a simple application, tested locally, tagged, uploaded to Docker Hub, and verified by pulling and running it from the repository

